Trees

Dr. Sayantan Panja

The concept of a Tree is one of the most important part in the study of Graph Theory and it is one of the most widely used subclasses of Graph. It was discovered by Cayley in 1857, who related his work on Trees to study chemical composition. Trees are widely used in computer science, mathematics, and real-life problems like organizing data, file systems, network designs, and decision-making processes.

1 Tree

Definition 1.1. *A connected graph without any cycle is called a tree.*

The following graph is an example of a tree.

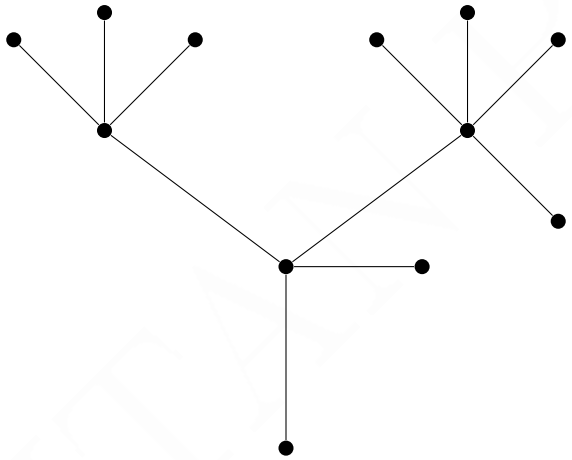


Fig. 1.1

Definition 1.2. *Let G be a graph. Then the* ***circumference*** *of G is denoted by $circ(G)$, is defined by the maximum length of a cycle in G. If G does not contain any cycle, then we define $circ(G) = 0$.*

Definition 1.3. *Let G be a connected graph with vertex set V,. For each $v \in V$, the* ***ecentricity*** *of V, denoted by $e(v)$, is defined by*

$$e(v) := \max\{d(u, v) : u \in V, u \neq v\}.$$

The vertex with least ecentricity in a graph G is known as the ***center*** *of the graph G.*

Definition 1.4. *Let G be a connected graph. Then the* ***radius*** *of G, denoted by $rad(G)$, is defined by $rad(G) := \min\{e(v) : e \in V\}$ and the* ***diameter*** *of G is defined by $diam(G) := \max\{e(v) : v \in V\}$. Thus, the diameter of G is given by*

$$diam(G) := \max\{d(u, v) : u, v \in G\}.$$

Definition 1.5. *A connected graph is said to be* ***minimally connected*** *if the graph becomes disconnected when one edge is removed.*

Note 1.6. *A minimally connected graph can not have a cycle. That is a minimally connected graph is a tree.*

Theorem 1.7. *A graph G is a tree if and only if there is one and only one path between every pair of vertices in G.*

Proof. Let G be a tree and let A, B be two arbitrary pair of vertices. Since G is a connected graph, A and B are connected by a path. If possible let, A and B be connected by two distinct paths. These two paths together form a cycle and then G cannot be a tree. So there is only one path connecting A and B.

Conversely, suppose there is a path between every pair of vertices. Then the graph G is connected. If possible let, G possess a cycle. So there exists at least one pair of vertices A, B such that there are two distinct paths between A and B. This contradicts the hypothesis. So G does not have any cycle. So, G is a tree. □

Theorem 1.8. *A tree T with n vertices has exactly $(n-1)$ edges.*

Proof. We prove this theorem by induction on n.

When $n = 1$, then T is a tree with 1 vertex. Since T has no loop, so it follows that T cannot have any edges, i.e., the number of edges is $0 = 1 - 1 = n - 1$. This establishes that the result is true for $n = 1$.

Now suppose that the result is true for $n = k$ where k is a positive integer. Now let T be a tree with $(k + 1)$ vertices where $k \geq 1$. Let u be a vertex of T of degree 1 (such a vertex exists definitely). Let $e = uv$ where v is the adjacent vertex of u. Then $T \setminus u$ is also a connected graph. We now show that $T \setminus u$ is acyclic.

If possible, let $T \setminus u$ contains a cycle and let C be a cycle in $T \setminus u$. Then C is also a cycle in T, a contradiction; since T is a tree. Thus the subgraph $T \setminus u$ is a tree with k vertices.

Hence by induction hypothesis $T \setminus u$ has $k - 1$ edges. Since $T \setminus u$ has exactly 1 edge less than T, it follows that T has k edges. Therefore, by the principle of mathematical induction, the result follows. □

Theorem 1.9. *A connected graph with n vertices and $n - 1$ edges is a tree.*

Proof. Let G be a connected graph with n vertices and $n - 1$ edges. Suppose, G be not a tree. Then G contains a cycle. Let e be an edge of this cycle. Then the subgraph $G - e$ is still connected.

Now, $G - e$ has $n - 2$ edges and n vertices. This is not possible since we know a connected graph with n vertices has at least $n - 1$ edges. So our assumption s wrong, and hence G is a tree. □

Theorem 1.10. *A graph is a tree if and only if it is minimally connected.*

Proof. First let T be a tree having n vertices. Then, T has $n - 1$ edges.

If one edge is removed from T, then it has $n - 2$ edges. Then T becomes disconnected, since a connected graph with n vertices must have at least $n - 1$ number of edges. Thus, deleting an edge makes the graph disconnected, and hence T is a minimally connected graph.

Conversely, let T be a minimally connected graph with n vertices. Since T is a connected graph, the number of edges of T is at least $n - 1$. Suppose, T be not a tree. Then T contains a cycle. After removing this cycle from T, it becomes still connected. This contradicts our hypothesis that T is a minimally connected graph. Hence, T is a tree. □

2 Binary Tree

Definition 2.1. *The tree in which there is exactly one vertex of degree 2 and each of the remaining vertices is of degree either 1 or 3 is called a binary tree.*

The following is an example of a binary tree.

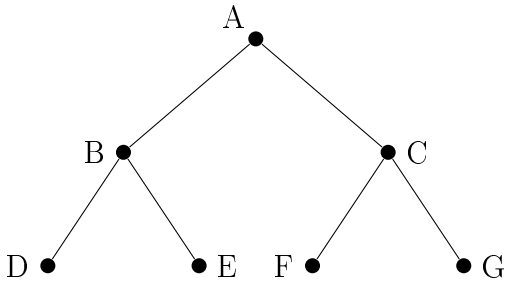


Fig. 2.1

Here, the vertex A is of degree 2; B, C are of degree 3 and D, E, F, G are of degree 1.

Note 2.2. *(i) The vertex of degree 2 in a binary tree is called the* ***root*** *of the tree.*

(ii) The vertex of a tree which is not pendent vertex is called the ***internal vertex****.*

Theorem 2.3. *The number of vertices in a binary tree is always odd.*

Proof. Let a binary tree has n vertices. Since a binary tree has one vertex of degree 2 and the remaining vertices are of degree either 1 or 3, so there are $(n-1)$ vertices of odd degrees.

We know that in a graph, there are even number of odd degree vertices. Therefore $(n-1)$ is an even integer. Hence n is an odd integer. □

Theorem 2.4. *Let T be a binary tree with n-vertices. Then the number of pendant vertex in T is* $\dfrac{n+1}{2}$.

Proof. Let x be the number of pendant vertices in the binary tree T, i.e., x is the number of vertices of degree one. Now T has only one vertex of degree 2 and each of the remaining vertices is of degree 3.

So, sum of the degrees of all vertices in T is $2+x+3(n-x-1) = 3n-2x-1$.

We know that in a graph, the sum of the degrees of all vertices $= 2\times$ number of edges.

Since the number of edges in a tree with n vertices is $(n-1)$, therefore,

$$3n-2x-1 = 2(n-1),$$

which implies that, $x = \frac{n+1}{2}$.

□

Theorem 2.5. *The number of internal vertices (i.e., non-pendant vertices) in a binary tree is one less than the number of pendant vertices.*

Proof. Let x be the number of pendant vertices and y be the number of non-pendant vertices (i.e., internal vertices) in the binary tree.

Therefore, the total number of vertices is $n = x+y$.

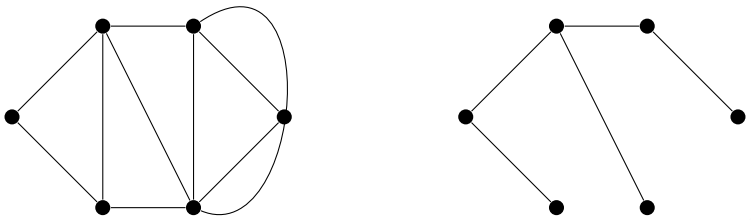
By above Theorem, $x = \frac{x+y+1}{2}$, implies, $y = x-1$.

Hence, number of internal vertices = number of pendant vertices -1. □

3 Spanning Tree

Definition 3.1. *A tree is called spanning tree of a connected graph G if T is a subgraph of G and T contains all the vertices of G.*

Example 3.2. *In the following graph, T is a spanning tree of the graph G.*



G T

Fig. 3.1

The following is an another example of a graph with its spanning tree.

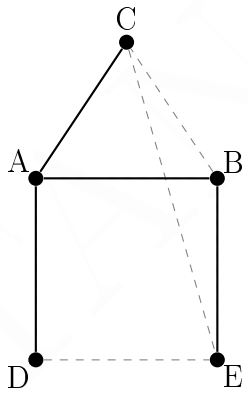


Fig. 3.2
A Graph with its Spanning Tree

Note 3.3. *(i) Spanning tree is the largest tree among all trees in a graph G. That's why it is also known as maximal tree within G.*

(ii) Spanning tree is also known as skeleton of the graph G.

Definition 3.4. *An edge of a tree is called a **branch** of the tree.*
*Let T be a tree contained in a graph G. An edge of G that is not in T is called **chord** of T in G.*

Theorem 3.5. *A graph G has a spanning tree if and only if G is connected.*

Proof. Let G be a connected graph. If G has no cycle, then it is its own spanning tree. If G contains a cycle, then delete an edge from that cycle. Then the remaining graph is still connected. If this graph has no cycle, then it becomes a spanning tree of G; otherwise, repeat the operation till an edge from the last cycle is removed; which gives a connected graph without any cycle. This new graph contains all vertices of G because no vertices were removed in the above deletion process. So this gives a spanning tree of G. Thus G has a spanning tree.

Conversely, let G be a graph having a spanning tree, say T. Let v_1 and v_2 be two arbitrary vertices of G. Since T contains all vertices of G, therefore v_1 and v_2 belong to T. Since T is a tree, so T is connected and so v_1 and v_2 are connected by a path. As v_1, v_2 are arbitrary, so any two vertices of G are connected by a path. So, G is connected. □

4 Minimal Spanning Tree

Sometimes a real number is associated with each edge of the graph. This number represents **weight** of the corresponding edge.

A graph each of whose edge bears a weights called weighted graph. Sum of the weights of all edges of the graph is called the weight of the graph.

Definition 4.1. *Let G be a weighted graph. A spanning tree with the smallest weight in G is called a* ***minimal spanning tree****.*

Example 4.2. *Find the minimal spanning tree of the following weighted graph G:*

G:

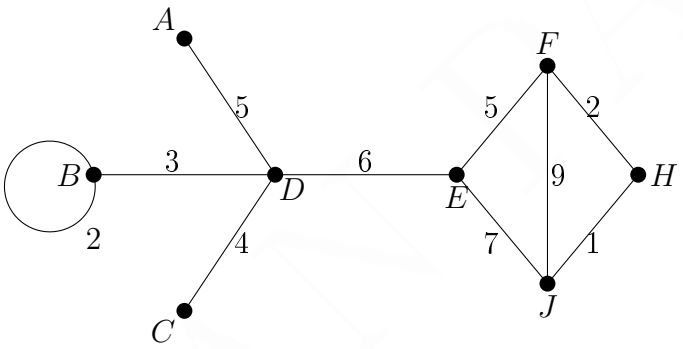


Fig. 4.1

Solution. All possible spanning trees of the graph G are as follows:

G_1:

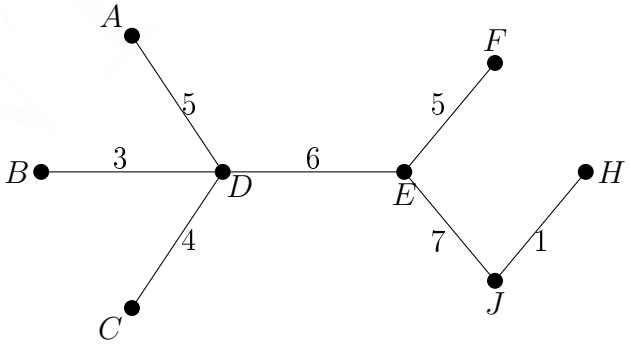


Fig. 4.2

G_2:

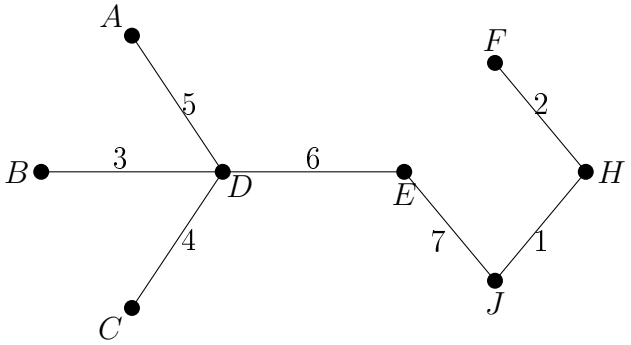


Fig. 4.3

G_3:

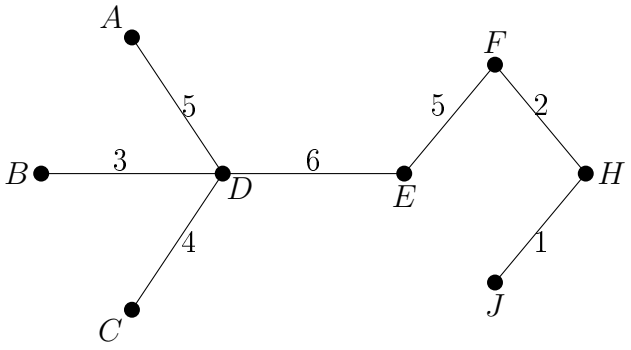


Fig. 4.4

G_4:

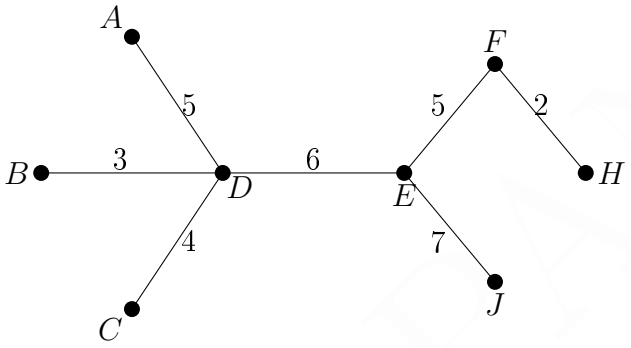


Fig. 4.5

G_5:

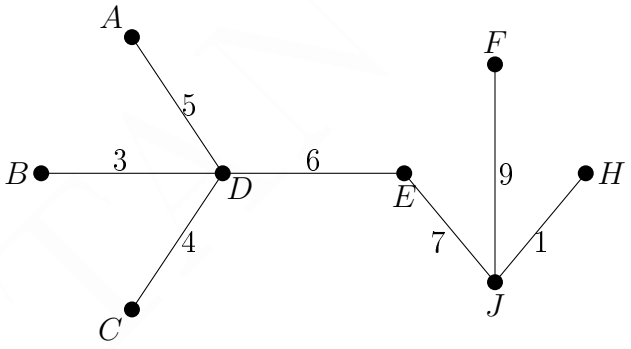


Fig. 4.6

G_6:

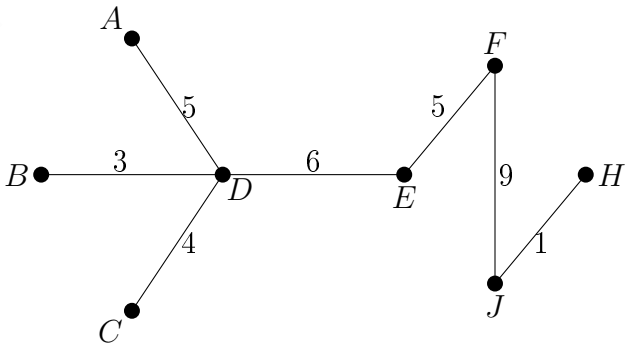


Fig. 4.7

Now, the weights of the spanning trees are as follows:

Spanning Tree	Weights
G_1	$5+3+4+6+5+7+1=31$
G_2	$5+3+4+6+7+1+2=28$
G_3	$5+3+4+6+5+2+1=26$
G_4	$5+3+4+6+5+7+2=32$
G_5	$5+3+4+6+7+9+1=35$
G_6	$5+3+4+6+5+9+1=33$

Therefore, the minimal spanning tree of the graph G is G_3.

Kruskal's Algorithm

Kruskal's algorithm is a popular method in graph theory used to find the Minimum Spanning Tree (MST) of a connected, weighted graph.

How Kruskal's Algorithm Works:

1. Remove all loops if exists; and delete all parallel edges between any two vertices except that having minimum weight. Then list all the remaining edges of the graph along with their weights.
2. Sort the edges in ascending order of weights.
3. Pick the smallest edge. If it does not form a cycle with the spanning tree formed so far, add it to the tree.
4. Repeat step 3 until there are exactly $(n-1)$ edges in the spanning tree (where n is the number of vertices).

Example 4.3. *Using Kruskal's Algorithm, find the minimal spanning tree of the following graph G.*

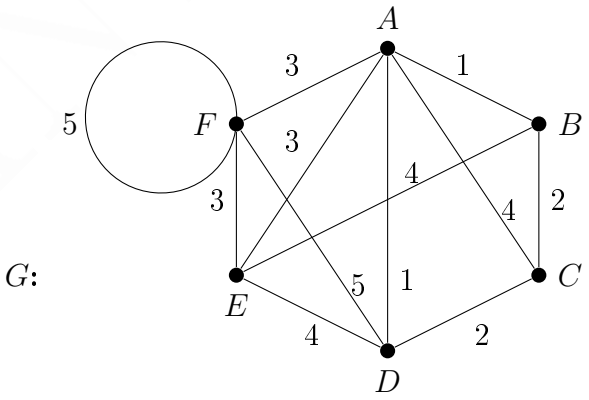


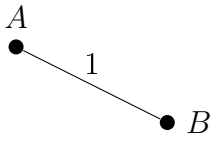
Fig. 4.8

Solution. Here, G contains 6 vertices. As a tree with n-vertices contains $(n-1)$-edges, so the shortest spanning tree of G contains 5 edges.

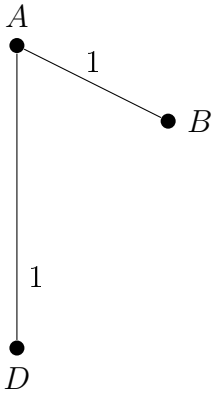
Step 1. List all the edges (except loop) and arrange in the order of non-decreasing weight.

Edges	AB	AD	BC	CD	AF	AE	FE	AC	EB	ED	FD
Weight	1	1	2	2	3	3	3	4	4	4	5

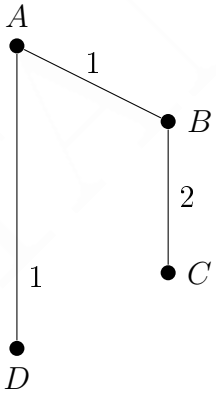
Step 2: First select the smallest edge AB.



Step 3: Select the next smallest edge AD. We accept it, as it does not form a cycle with AB.

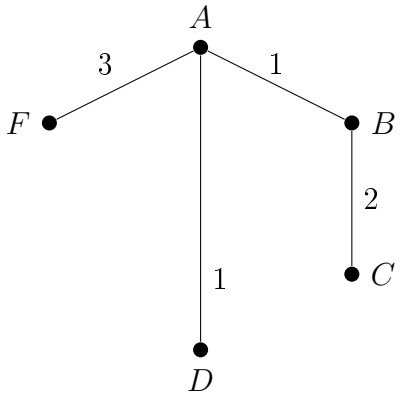


Step 4: Select the next smallest edge BC. As it does not form any cycle with DAB, so we accept it.

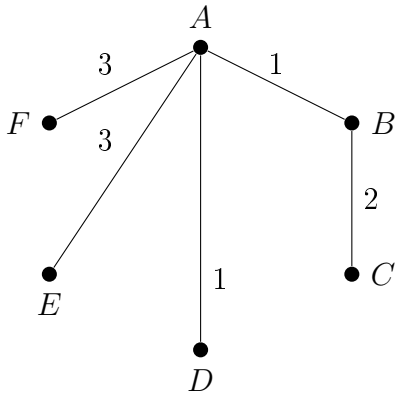


Step 5: Select the next smallest edge CD. As it forms a cycle with $DABC$, so we reject it and go to next step.

Step 6: Select the next smallest edge AF. As it does not form any cycle with $DABC$, we accept it.



Step 7: Select the next smallest edge AE. We accept it, as it does not form any cycle with the last graph.



Since, this is a tree containing 5 edges, so we stop the process and this is the required smallest spanning tree. The weight of this spanning tree is 1+1+2+3+3 = 10.

Note 4.4. *(i) Minimal spanning tree need not necessarily be unique. There exists other minimal spanning tree. For example, if we choose FE in step-7 instead of AE, then we get another minimal spanning tree.*

(ii) Weight of every minimal spanning tree will be same.

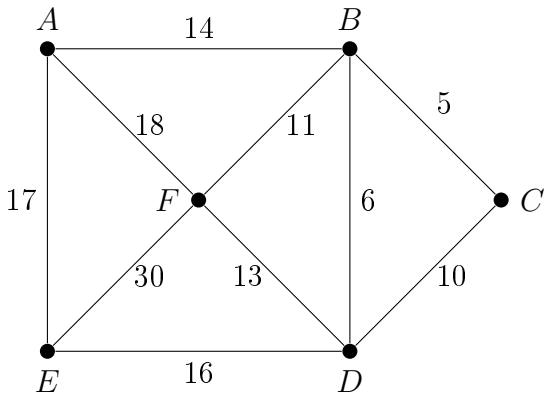
Prim's Algorithm

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted graph.

How Prim's Algorithm Works:

1. Start with any vertex (usually arbitrarily chosen).
2. Select the smallest weight edge that connects a vertex in the tree to a vertex outside the tree.
3. Add the selected edge and vertex to the tree.
4. Repeat step 2 until all vertices are included in the tree.

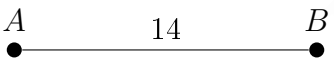
Example 4.5. *Using Prim's algorithm, find a minimal spanning tree for the following graph:*



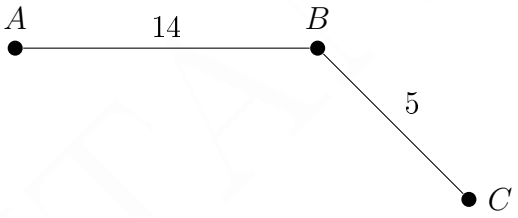
Solution. The given graph is connected weighted graph having 6 vertices, so the minimal spanning tree of this graph has 5 edges. The tabulated form of the weights of its edges are given below:

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} 0 & 14 & \infty & \infty & 17 & 18 \\ 14 & 0 & 5 & 6 & \infty & 11 \\ \infty & 5 & 0 & 10 & \infty & \infty \\ \infty & 6 & 10 & 0 & 16 & 13 \\ 17 & \infty & \infty & 16 & 0 & 30 \\ 18 & 11 & \infty & 13 & 30 & 0 \end{bmatrix} \end{array}$$

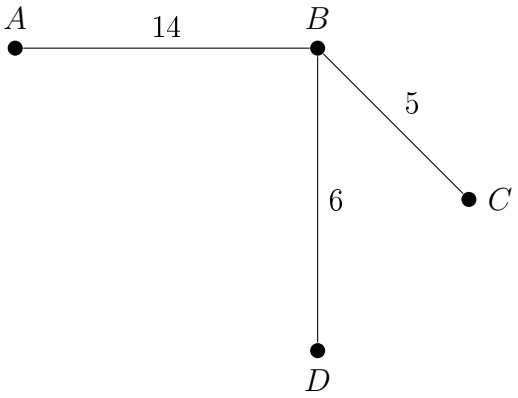
Step 1: Start from the vertex A. The smallest entry in A-row (i.e., in 1st row) is 14 (except 0, otherwise it will form self loop), which corresponds to the vertex B. Thus the following tree is formed.



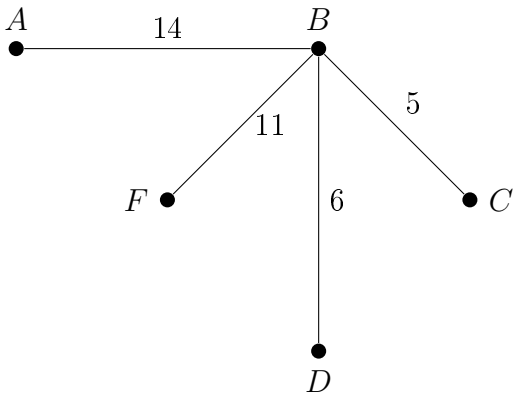
Step 2: Smallest entry in A-row and B-row is 5, which corresponds to the vertex C. Thus the following tree is formed.



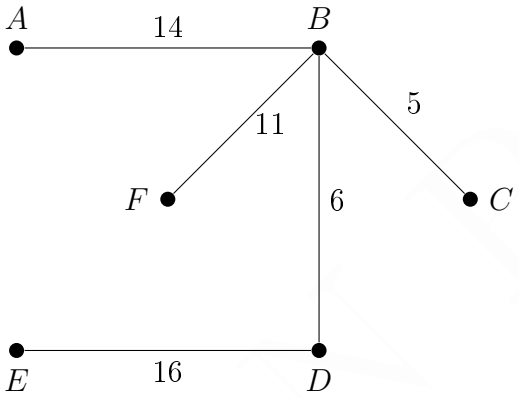
Step 3: Smallest entry in A-row, B-row and C-row is 6, which corresponds to the vertex D (in the B-row). Thus the following tree is formed.



Step 4: Smallest entry in A-row, B-row, C-row and D-row is 11, which corresponds to the vertex F. Thus the following tree is formed.



Step 5: The smallest entry in A-row, B-row, C-row, D-row and E-row is 16, that corresponds to the vertex D. Thus the following tree is formed.



Since this tree has 6-vertices and 5-edges, so we stop the process and this is the required minimal spanning tree.

Example 4.6. *The following table shows the distances in kilometers, between six villages in India. Find minimal spanning tree connecting the six villages applying Kruskal's algorithm, as well as Prim's algorithm.*

$$\begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \end{array} \begin{array}{c} \begin{array}{cccccc} A & B & C & D & E & F \end{array} \\ \begin{bmatrix} 0 & 5 & 6 & 12 & 4 & 7 \\ 5 & 0 & 11 & 3 & 2 & 5 \\ 6 & 11 & 0 & 8 & 6 & 6 \\ 12 & 3 & 8 & 0 & 7 & 9 \\ 4 & 2 & 6 & 7 & 0 & 8 \\ 7 & 5 & 6 & 9 & 8 & 0 \end{bmatrix} \end{array}$$